When Do Multiscale Features Help Small Defect Localization? A Study of Accuracy and Efficiency in PatchCore

Final Project Proposal

Members of Group 6: Yang Cao, Bernie Chen, Celine Li

Project Description

Small scratches, cracks, and surface defects are often hard to localize in industrial images. In convolutional neural networks (CNNs), intermediate layers keep fairly fine spatial detail, while deeper layers capture more abstract visual information. PatchCore already combines features from more than one layer for unsupervised anomaly detection[1]. We are not proposing a new architecture. Instead, we want to find out **when this multiscale fusion actually helps with small defects, and whether the benefit is worth the extra memory and inference time.**

We will test three hypotheses:

- **H1:** layer2 features localize small defects better than layer3 features, since they have a finer spatial representation.
- **H2:** combining layer2 and layer3 improves localization over either layer alone, although the size of the gain may depend on the defect type and object category.
- **H3:** the advantage of fusion may shrink when the configurations are compared at similar memory budgets. We treat these as open questions, not expected results.

We will use the **screw, capsule, and metal_nut** categories from **MVTec AD**[2]. We will keep the official training and test split and train a separate model for each category, using only normal training images to build the memory bank. Before evaluating any model, we will go through the ground truth masks and label an image as a *small defect image* if its total annotated defect area is at most **1% of the original image area**. We will report sample counts and defect area distributions for each category. Since defect area alone does not capture how hard a defect is to see, we will also look at contrast, shape, and background texture as possible reasons for failure cases. We will also check whether preprocessing crops out any annotated defect pixels.

Following the official PatchCore implementation[3], we will freeze an **ImageNet pretrained WideResNet50** and compare three setups: **layer2 only**, **layer3 only**, and **layer2 plus layer3**, which is the original PatchCore baseline. All main runs will share the same preprocessing (**resize to 256, then center crop to 224**), neighborhood aggregation, target embedding dimension, nearest neighbor scoring, and **10% approximate greedy coreset sampling**. Ground truth masks will go through the same spatial transform using nearest neighbor interpolation. We will confirm the resulting feature map resolutions and memory bank sizes, because the same sampling percentage does not always mean the same

memory cost. If time allows, a secondary experiment will match memory budgets across the three setups.

Our main localization metric will be **AUPRO**, along with **pixel level AUROC** and **image level AUROC**. Results for small and larger defects will be reported separately, but we will be careful when interpreting subgroup differences with small counts. We will not tune thresholds or hyperparameters on the official test masks. To check robustness, we will repeat coreset sampling with **three fixed random seeds** and, where sample sizes allow, compute paired bootstrap confidence intervals at the image level. On the same hardware, we will also measure memory bank storage and median inference latency after a warm up period.

The final report will include **defect area histograms**, performance tables for each category, **plots of the trade off between accuracy and memory**, and side by side views of input images, ground truth masks, and anomaly heatmaps. Together, these should show whether shallow features, deeper features, or their combination give the best practical balance for finding small defects.

References

1. Roth, K., et al. (2022). *Towards Total Recall in Industrial Anomaly Detection*. CVPR. https://arxiv.org/abs/2106.08265
2. Bergmann, P., et al. (2019). *MVTec AD: A Comprehensive Real World Dataset for Unsupervised Anomaly Detection*. CVPR. https://www.mvtec.com/research-teaching/datasets/mvtec-ad
3. Official PatchCore code and example configuration: https://github.com/amazon-science/patchcore-inspection